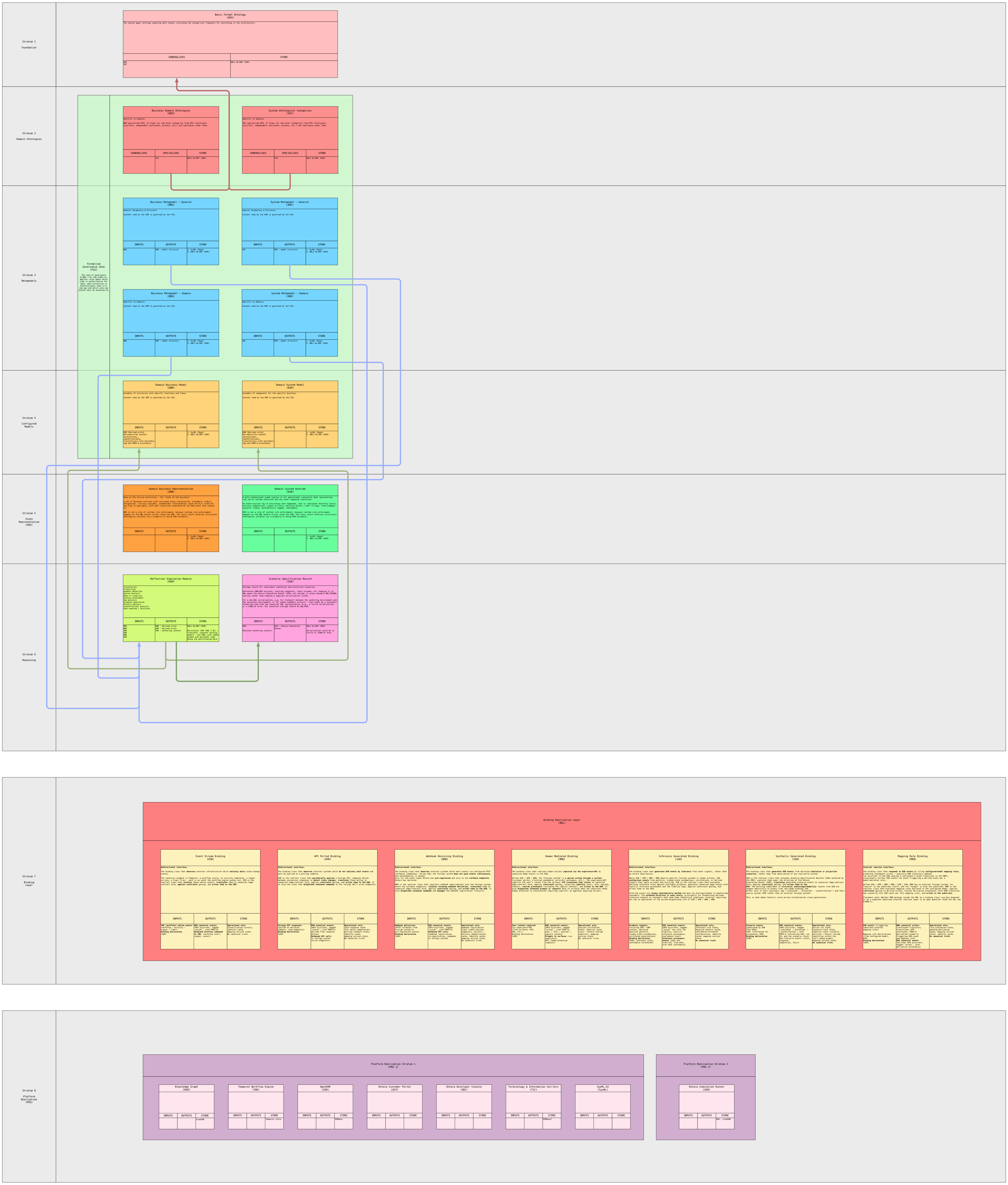


## Stratum 1
Foundation

### Basic Formal Ontology
(BFO)

The shared upper ontology spanning both stacks, providing the categorical framework for everything in the architecture.

| GENERALISES | STORE |
|---|---|
| BDO<br>SOC | OWL2 DL/RDF (KGR) |

## Stratum 2
Domain Ontologies

**Formalism Governance Zone (FGZ)**

The zone of governance forOWL 2 DL and SysML.v2. Applies rules about which side is authoritative for what, when projection is bidirectional, when it's one-way and which side new content must be authored on.

### Business Domain Ontologies
(BDO)

Specific to domains.

BDO specialiseS BFO. It draws its top-level categories from BFO (continuant, occurrent, independent continuant, process, etc.) and subclasses under them.

| GENERALISES | SPECIALISES | STORE |
|---|---|---|
|  | FGZ | OWL2 DL/RDF (KGR) |

### System Ontological Categories
(SOC)

Specific to domains.

SOC specialiseS BFO. It draws its top-level categories from BFO (continuant, occurrent, independent continuant, process, etc.) and subclasses under them.

| GENERALISES | SPECIALISES | STORE |
|---|---|---|
|  | FGZ | OWL2 DL/RDF (KGR) |

## Stratum 3
Metamodels

### Business Metamodel – General
(BMG)

General Vocabulary & Structure

Content read by the RSM is governed by the FGZ.

| INPUTS | OUTPUTS | STORE |
|---|---|---|
| BDO | RSM – model structure | 1. SysML (Repo)<br>2. OWL2 DL/RDF (KGR) |

### System Metamodel – General
(SMG)

General Vocabulary & Structure

Content read by the RSM is governed by the FGZ.

| INPUTS | OUTPUTS | STORE |
|---|---|---|
| SOC | RSM – model structure | 1. SysML (Repo)<br>2. OWL2 DL/RDF (KGR) |

### Business Metamodel – Domain
(BMD)

Specific to domains.

Content read by the RSM is governed by the FGZ.

| INPUTS | OUTPUTS | STORE |
|---|---|---|
| BDO | RSM – model structure | 1. SysML (Repo)<br>2. OWL2 DL/RDF (KGR) |

### System Metamodel – Domain
(SMD)

Specific to domains.

Content read by the RSM is governed by the FGZ.

| INPUTS | OUTPUTS | STORE |
|---|---|---|
| SOC | RSM – model structure | 1. SysML (Repo)<br>2. OWL2 DL/RDF (KGR) |

## Stratum 4
Configured Models

### Domain Business Model
(DBM)

Assembly of structures with specific functions and flows.

Content read by the RSM is governed by the FGZ.

| INPUTS | OUTPUTS | STORE |
|---|---|---|
| RSM (Derived write) Derived-write content (projections, counterfactuals, trajectories) with epistemic tag and PROV-O provenance |  | 1. SysML (Repo)<br>2. OWL2 DL/RDF (KGR) |

### Domain System Model
(DSM)

Assembly of components for the specific business.

Content read by the RSM is governed by the FGZ.

| INPUTS | OUTPUTS | STORE |
|---|---|---|
| RSM (Derived write) Derived-write content (projections, counterfactuals, trajectories) with epistemic tag and PROV-O provenance |  | 1. SysML (Repo)<br>2. OWL2 DL/RDF (KGR) |

## Stratum 5
State Representation (SRS)

### Domain Business Representation
(DBR)

Data on the service activities – the 'state of the business'

A set of business entities with versioned state trajectories. Customers, orders, obligations, clinical episodes, stakeholder relationships. Each entity's state-at-any-time is queryable, with each transition annotated by the DSR event that caused it.

DBR is not a site of runtime rule enforcement, because runtime rule enforcement happens at the BRL before writes reach the DBR. The store itself enforces structural/ontological validity (as a property of being KGR-resident).

| INPUTS | OUTPUTS | STORE |
|---|---|---|
|  |  | 1. SysML (Repo)<br>2. OWL2 DL/RDF (KGR) |

### Domain System Runtime
(DSR)

A multi-dimensional graph capture of all operational simulation data representing real world runtime execution and any other computed simulation.

An event-sourced log of everything that happened, real or simulated: Workflow starts, activity completions, signal arrivals, decision points, timer firings. Time-stamped, causally linked, epistemically tagged, replayable.

DSR is not a site of runtime rule enforcement, because runtime rule enforcement happens at the BRL before writes reach the DSR. The store itself enforces structural/ontological validity (as a property of being KGR-resident).

| INPUTS | OUTPUTS | STORE |
|---|---|---|
|  |  | 1. SysML (Repo)<br>2. OWL2 DL/RDF (KGR) |

## Stratum 6
Reasoning

### Reflection Simulation Module
(RSM)

Trajectories
Projections
Anomaly detection
Option analysis
What-if scenarios
Valence assessment
Gap analysis
Guidance generation
History analysis
Counterfactual analysis
Goal-seeking / solutions

| INPUTS | OUTPUTS | STORE |
|---|---|---|
| BMG<br>BMD<br>SMG<br>SMD<br>DBR<br>DSR | DBM – derived write<br>DSM – derived write<br>SSR – authoring content | OWL2 DL/RDF (KGR)<br><br>Persistent: KGR (OWL 2 DL) / Transient: reasoner working memory – one KGR, with named graphs and epistemic tags doing the partitioning work |

### Scenario Specification Record
(SSR)

Storage record for individual simulation and projection scenarios.

References DBM/DSM versions, starting snapshots, input streams, etc. Keeping it in OWL means the Ontara Simulation Runner (OSR) can consume it using standard OWL/SPARQL tooling rather than needing a separate serialisation format.

For a non-OWL serialisation, e.g. for transport between the authoring environment and the execution environment, or for human-readable archival – that might be a secondary format derived from the canonical OWL representation (e.g., a Turtle serialisation, or a JSON-LD form). But canonical storage should be OWL/KGR.

| INPUTS | OUTPUTS | STORE |
|---|---|---|
| RSM<br><br>Receives authoring content. | OSR – Ontara Simulation Runner | OWL2 DL/RDF (KGR)<br><br>Serialisation could be in Turtle or JSON-LD form. |

## Stratum 7
Binding Layer

### Binding Realisation Layer
(BRL)

#### Event Stream Binding
(ESB)

**Bidirectional interface:**

The binding class that **observes** external infrastructure which **natively emits** state-change events.

The canonical example is Temporal: a workflow starts, an activity completes, a signal arrives, a timer fires – each is an event the workflow engine pushes out. ESB is the realiser class that **receives** those native events, **translates** them to canonical edge-contract form, **applies constraint** gating, and **writes them to the DSR.**

| INPUTS | OUTPUTS | STORE |
|---|---|---|
| **TWE (workflow) native events:** (workflow / activity / signal / timer)<br>**Binding declaration** (KGR) | **DSR canonical events:** (BFO occurrent, tagged `actual`, with PROV-O)<br>**Outbound workflow commands to TWE:** (workflow start, signal, cancel) | **Operational only:** reconciliation cursors, identity cache, mapping version state. No canonical truth. |

#### API Polled Binding
(APB)

**Bidirectional interface:**

The binding class that **observes** external systems which **do not natively emit events** and must be queried on a polling cadence.

APB is the realiser class that **periodically queries** a foreign API, computes deltas between successive responses to **detect state changes, translates** those deltas to canonical edge-contract form, applies **constraint** gating, and **writes them to the DSR.** It is also the class that **dispatches outbound commands** to the foreign API's write endpoints.

| INPUTS | OUTPUTS | STORE |
|---|---|---|
| **Foreign API responses:** (polled on declared cadence, read endpoints)<br>**Binding declaration** (KGR) | **DSR canonical events:** (BFO occurrent, tagged `actual`, with PROV-O; derived from response deltas)<br>**Outbound API calls:** to foreign system (write endpoints) | **Operational only:** last-response cache (for delta computation), polling schedule state, identity cache, mapping version state. No canonical truth. |

#### Webhook Receiving Binding
(WRB)

**Bidirectional interface:**

The binding class that **observes** external systems which emit events via configured HTTP callbacks (webhooks). Unlike ESB, the foreign system **does not push events continuously** and autonomously; it pushes only for the event types Ontara has **pre-registered** and only to the **callback endpoints** Ontara has declared.

WRB is the realiser class that registers webhook subscriptions with the foreign system, hosts the callback endpoints, **receives incoming webhook deliveries, translates** them to canonical edge-contract form, applies **constraint** gating, and **writes them to the DSR.** It also **dispatches outbound commands** and **manages** the webhook registration lifecycle.

| INPUTS | OUTPUTS | STORE |
|---|---|---|
| **Webhook deliveries:** (HTTP callbacks from foreign system on pre-registered events)<br>**Binding declaration** (KGR) | **DSR canonical events:** (BFO occurrent, tagged `actual`, with PROV-O)<br>**Outbound API calls** (webhook registration, re-registration, commands to foreign system) | **Operational only:** webhook registration state (subscriptions, endpoints, secrets), delivery deduplication cache, identity cache, mapping version state. No canonical truth. |

#### Human Mediated Binding
(HMB)

**Bidirectional interface:**

The binding class that realises human actions **captured via the experience-API** as canonical-edge events in the DSR.

Unlike ESB / APB / WRB, the "foreign system" is **a person acting through a surface** (customer portal, clinician dashboard, architect workspace, etc.). The experience-API captures the user's intent from the surface; HMB **translates that intent** to canonical edge-contract form, applies **constraint** gating, reconciles **identity** (user, role, session, tenant), **records provenance** (including the capture context), and **writes to the DSR.** It also **dispatches outbound prompts or requests** back to surfaces when the substrate needs human attention (a consultation requiring sign-off, an approval awaiting action).

| INPUTS | OUTPUTS | STORE |
|---|---|---|
| **User intents captured** via experience-API (from surfaces: OCP, ODC, etc.)<br>Binding declaration (KGR) | **DSR canonical events:** (BFO occurrent, tagged `actual`, with PROV-O; user / role / session / capture context)<br>**Prompts to surfaces** (via experience-API; human-attention requests) | **Operational only:** session correlation state, identity cache (user → Ontara runtime identity), mapping version state. No canonical truth. |

#### Inference Generated Binding
(IGB)

**Bidirectional interface:**

The binding class that **generates DSR events by inference** from other signals, rather than by direct observation.

Where ESB / APB / WRB / HMB observe specific foreign systems or human actions, IGB **synthesises events** from patterns, steady-state assumptions, correlations, or derived signals that imply an event has occurred. IGB is the realiser class that evaluates declared inference rules against available evidence, generates canonical-edge events with explicit inference provenance and low fidelity tags, applies constraint gating, and writes them to the DSR.

Inferred events are **always epistemically marked** and must be distinguishable to downstream consumers. The **outbound direction is used rarely**, principally for dispatching derived-inference outputs to consumers that need them (analytical pipelines, external reporting) but has no equivalent of the action-dispatching role of ESB / APB / WRB / HMB.

| INPUTS | OUTPUTS | STORE |
|---|---|---|
| **Evidence signals:** (existing DSR / DBR content, declared baseline patterns, steady-state assumptions, correlated observations)<br>**Binding declaration** (KGR; inference rules, confidence thresholds) | **DSR canonical events:** (BFO occurrent, tagged `actual` but with low fidelity, explicit inference provenance, confidence level)<br>**Inference provenance events** (what was inferred, from what, with what confidence) | **Operational only:** inference rule state, baseline pattern cache, confidence-threshold configuration, identity cache, mapping version state.<br>**No canonical truth.** |

#### Synthetic Generated Binding
(SGB)

**Bidirectional interface:**

The binding class that **generates DSR events** from declared **simulation or projection scenarios,** rather than from observation of any real-world system.

SGB is the realiser class that consumes Scenario Specification Records (SSRs authored by the RSM), executes them under the direction of the Ontara Simulation Runner (OSR), **translates** generated scenario events to canonical edge-contract form, applies **constraint** gating, and **writes them to the DSR.** The defining commitment is **structural indistinguishability:** events from SGB are shaped identically to events from real-mode bindings and differ only in their epistemic tag (`simulated`, `projected`, `counterfactual`) and their source system (OSR rather than an external foreign system).

This is what makes Ontara's unity-across-instantiation claim operational.

| INPUTS | OUTPUTS | STORE |
|---|---|---|
| **Scenario events** (generated by OSR from SSRs)<br>SSRs (referenced for provenance, KGR)<br>**Binding declaration** (KGR) | **DSR canonical events:** (BFO occurrent, tagged `simulated` / projected` / `counterfactual`, with PROV-O referencing SSR, run ID, and the scenario clock)<br>Run-lifecycle events (start, progress, completion, fault) | **Operational only:** active run state, scenario-clock state, identity cache (scenario entities → Ontara runtime identities within the run's named graph), mapping version state.<br>**No canonical truth.** |

#### Mapping Rule Binding
(MRB)

**Internal reaction interface:**

The binding class that **responds to DSR events** by firing **configured-model mapping rules,** producing consequent writes – typically DBR trajectory updates (an occurrent in the DSR causing a continuant's state transition in the DBR), occasionally further DSR events (an event triggering a derived event per a model-declared rule).

Unlike ESB / APB / WRB / HMB / IGB / SGB, MRB has no external foreign system; its "source" is the substrate itself, and its "target" is also the substrate. MRB is the realiser class that evaluates mapping rules declared in the configured model, applies **constraint** gating to derived writes, records derivation provenance (this DBR transition was caused by this DSR event per this mapping rule), and **writes to the substrate.**

Placement note: Whether MRB belongs inside the BRL as a binding class (current diagram) or as a separate substrate-internal reaction layer is an open question (S234 §12 Q4; see Q-MRB-1).

| INPUTS | OUTPUTS | STORE |
|---|---|---|
| **DSR events** (triggering substrate-internal mapping rules)<br><br>Mapping rule declarations (from configured model, KGR)<br>**Binding declaration** (KGR) | **DBR canonical writes:** (continuant trajectory transitions; BFO continuant, PROV-O derivation-linked to triggering DSR event and mapping rule)<br>**DSR canonical events** (derived; BFO occurrent, tagged `actual`, with derivation provenance) | **Operational only:** rule evaluation state, pending-derivation queue, mapping version state, identity cache.<br>**No canonical truth.** |

## Stratum 8
Platform Realisation (PRS)

### Platform Realisation Stratum 1
(PRS 1)

#### Knowledge Graph
(KGR)

| INPUTS | OUTPUTS | STORE |
|---|---|---|
|  |  | GraphDB |

#### Temporal Workflow Engine
(TWE)

| INPUTS | OUTPUTS | STORE |
|---|---|---|
|  |  | Temporal store |

#### OpenEHR
(CDR)

| INPUTS | OUTPUTS | STORE |
|---|---|---|
|  |  | EHRBase |

#### Ontara Customer Portal
(OCP)

| INPUTS | OUTPUTS | STORE |
|---|---|---|
|  |  |  |

#### Ontara Developer Console
(ODC)

| INPUTS | OUTPUTS | STORE |
|---|---|---|
|  |  |  |

#### Terminology & Information Carriers
(TIC)

| INPUTS | OUTPUTS | STORE |
|---|---|---|
|  |  | EHRBase? |

#### SysML.V2
(SysML)

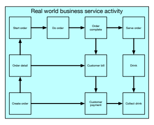
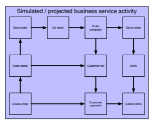
| INPUTS | OUTPUTS | STORE |
|---|---|---|
|  |  |  |

### Platform Realisation Stratum 2
(PRS 2)

#### Ontara Simulation Runner
(OSR)

| INPUTS | OUTPUTS | STORE |
|---|---|---|
|  |  | KGR – GraphDB |

Ontara Platform
v3.0.0
10/04/2026

(Draft)